Work with Docker

Practical

How to Push Docker Image to Docker Hub

To push a Docker image to Docker Hub, you can follow these steps:

1. **Create a Docker Hub Account:**

   To proceed, visit https://hub.docker.com and register for a Docker Hub account if you don't already have one.

   Creating an account will grant you a username and password that you can use for authentication purposes.

2. **Tag the Docker Image:** Before performing a Docker Push, it's essential to ensure that your Docker image is properly tagged. You can assign a tag to an existing image using the following command:

```
docker tag <image-id> <docker-hub-username>/<image-name>:<tag>
```

- Replace `<image-id>` with the ID or name of your local Docker image. `<docker-hub-username>` should be your Docker Hub username, `<image-name>` is the name you want to give to your image, and `<tag>` represents a specific version or label for the image.

3. **Log in to Docker Hub:** Use the following command to log in to your Docker Hub account from the command line:

- **docker login**
- You will be prompted to enter your Docker Hub username and password.

4. **Push the Docker Image:** Once logged in, push the tagged Docker image to Docker Hub using the command:

```
docker push <docker-hub-username>/<image-name>:<tag>
```

Replace `<docker-hub-username>`, `<image-name>`, and `<tag>` with the respective values you used

- 1. firstly find the images that you want to push in respository

- 2. Then use the login command to get login in docker hub as:

```
$ docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Do
cker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grants better
 security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-toke
ns/

Username: harpreet81
Password:
WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

3. Then use tag command to give tag name to image :

Docker tag <image name> <tag name>

```
[node1] (local) root@192.168.0.28 ~
$ docker tag python harpreet81/sample
[node1] (local) root@192.168.0.28 ~
$ docker images
REPOSITORY          TAG       IMAGE ID       CREATED       SIZE
harpreet81/sample   latest    86ac1e3337a9   6 weeks ago   1.02GB
python              latest    86ac1e3337a9   6 weeks ago   1.02GB
```

- 4. use push command

- Docker push

```
$ docker push harpreet81/sample
Using default tag: latest
The push refers to repository [docker.io/harpreet81/sample]
d08e8c3991bb: Mounted from library/python
00fdc842d356: Mounted from library/python
a2ad80a8ed2e: Mounted from library/python
74ec933aa24e: Mounted from library/python
d43f876e6f21: Mounted from library/python
9f843c569746: Mounted from library/python
bcd354c940e1: Mounted from library/python
1dae5147cd29: Mounted from library/python
latest: digest: sha256:97a22f3bb3ab8d549ff1355d80d0c2b016f0c0e1dd1adf4d90bc8142038cf4ac size: 2007
```

Ex#2(Practice Question):

1.Update the ubuntu container, install apache and php, then push updated on to docker hub.

2. Compare the size of base image and updated image after commit

- 1. docker run –d –it –name a1 ubuntu
- 2. docker exec –it a1 bash
- 3. #: apk update
- # apk add apache2
- # apk add php
- # exit
- 4. docker commit a1 harpreet81/ub-a:v1
- 5. docker login
- 6. docker push harpreet81/alpine-a:v1

To Build your own image from base image

Let's learn the concept of DOCKERFILE

To Create Docker Image by Docker file

Dockerfile:

- Dockerfile is a simple text file that consists of instructions to build Docker images.
- Mentioned below is the syntax of a Dockerfile:

Syntax

- *# comments*
- *command argument argument1...*
- Example
- *# Print "Get Certified. Get Ahead"*
- *Run echo "Get Certified. Get Ahead"*

The specific commands you can use in a dockerfile are

FROM, PULL, RUN, and CMD

- FROM - Creates a layer from the ubuntu:18.04
- PULL - Adds files from your Docker repository
- RUN - Builds your container
- CMD - Specifies what command to run within the container

ENTRYPOINT allows specifying a command along with the parameters

ADD command helps in copying data into a Docker image

ADD /[source]/[destination]

ENV provides default values for variables that can be accessed within the container

ENV key value

Ex#1 : Create Ubuntu docker file

- # Base Image
- FROM ubuntu:latest

- # Maintainer (optional)
- LABEL maintainer="Student Demo"

- # Install package
- RUN apt update && apt install -y php

- # Default command when container runs
- CMD ["echo", "Hello Students"]

Ex#2: Create Java Docker file

1. $ mkdir java-docker
2. **Create a Java File**

- Now create a Java file. Save this file as **Hello.java** file.
- **// Hello.java**
- **class** Hello{
- **public static void** main(String[] args){
- System.out.println("This is java app \n by using Docker");

- }
- }

Save it inside the directory **java-docker as hello.java**

- ## 3. Create a Dockerfile

```
FROM java
COPY . /var/www/java
WORKDIR /var/www/java
RUN javac Hello.java
CMD ["java", "Hello"]
```

- ## 4. Build Docker Image

After creating Dockerfile, we are changing working directory.

- $ cd java-docker-app

5. $ docker build -t java-app .
6. $ docker run java-app

- FROM java:8
- COPY . /var/www/java
- WORKDIR /var/www/java
- RUN javac Hello.java
- CMD ["java", "Hello"]

Few more commands in dockerfile:

- WORKDIR Command

- Sets the **default directory inside container**
  If directory does NOT exist → Docker creates it
- All next commands execute inside this folder

- COPY Command

- Copies files/folders from **host** → **container**

- Simple file transfer

- No extra processing

- ADD Command

- Copies files
- **Auto extracts archives (.tar.gz)**
- **Downloads files from URLs**

- RUN Command

- Executes commands **during image build**
- Creates new Docker layer
- Used for installations & setup

Summary

- **WORKDIR** → "Where work happens"
- **COPY** → "Move files"
- **ADD** → "Move files + magic"
- **RUN** → "Execute commands while building image"

Docker file#3 for Python application

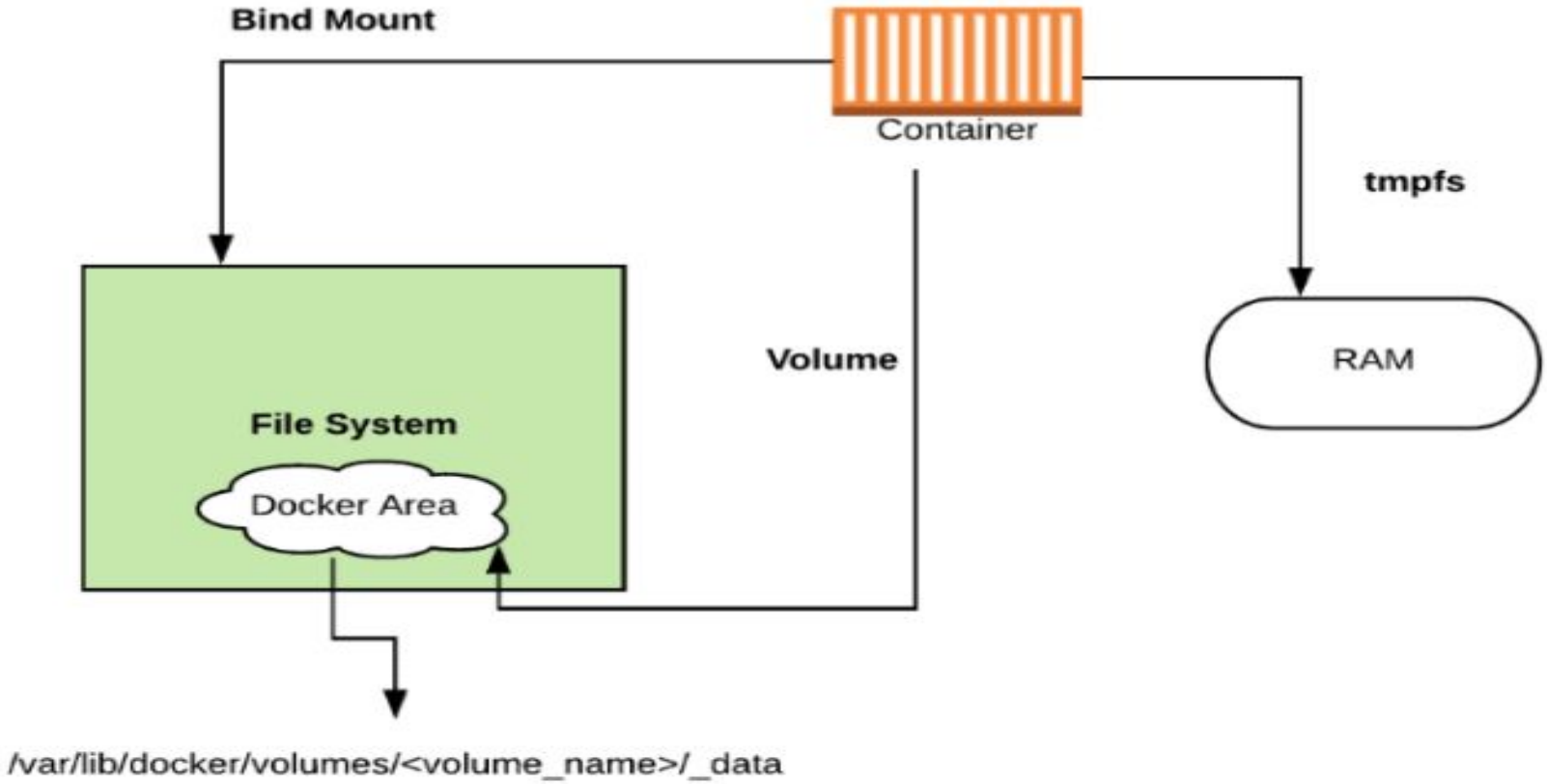
- # 1. Base image
- FROM python:3.11-slim
- # 2. Set working directory
- WORKDIR /app
- # 3. Copy dependency file
- COPY requirements.txt
- .# 4. Install dependenciesR
- UN pip install --no-cache-dir -r requirements.txt
- # 5. Copy application code
- COPY . .
- # 6. Expose port
- EXPOSE 5000
- # 7. Run the app
- CMD ["python", "app.py"]

App.py

```
from flask import Flask

app = Flask(_name_)

@app.route("/")
def home():
    return "Hello from Docker!"

app.run(host="0.0.0.0", port=5000)
```

Types of Storage in Docker

Docker Storage Types ^



Docker storage distinguishes **three storage** types.

Two types are permanent: **Docker volumes** and **bind Mounts**

and the third way of writing data is **tmpfs**.

From the container perspective, it **doesn't know** what sort of storage is in use.

The difference between these is, **volumes** have a dedicated filesystem on the host (/var/lib/ docker/volumes) and are directly controlled through the Docker CLI.

On the other hand, **bind mounts** use any available host filesystem.

Whereas **tmfs**, uses the host memory.

Docker Volume

- Docker volume is the most commonly used technology for the **permanent storage** of container data.

- Docker volume is managed by Docker itself and has a dedicated filesystem on the host, doesn't depend upon the filesystem structure on the host.

- Docker volumes are explicitly managed via the Docker command line and can be created alone or during container initialization. T

- he command used is

docker volume create.

- When stopping or deleting a container, Docker volume remains permanently stored.

- The volumes are often manually deleted with the docker volume prune command.

- Multiple containers can be connected to the same Docker volume.

Commands used for the Volumes

1. docker volume create

Used to Create a volume

2. docker volume inspect

Display detailed information on one or more volumes

3. docker volume ls

List volumes

4.docker volume prune

- Remove all unused local volumes

5. docker volume rm

- Remove one or more volumes

Some use cases for volumes include:

- Sharing data among multiple running containers.
- Volume is helpful if the Docker host is not guaranteed to have a given directory or file structure.
- When you want to store your container's data on a remote host or a cloud provider instead of storing locally.
- Volumes are helpful if you need to back up, restore, or migrate data from one Docker host to another, volumes are a better choice

Docker Bind Mount

- Docker bind mount is the second **permanent storage** option but with more limited options than Docker volume.

- It can't be managed via Docker CLI and is totally dependent on the availability of the filesystem of the host.

- A host filesystem can be created when running a container. Bind mounts are a sort of **superset of Volumes (named or unnamed)**.

tmpfs Mounts

tmpfs is a third storage option that is **not permanent** like Docker volume or bind mount.

The data is written directly on to the host's memory and deleted when the container is stopped.

Very useful when it involves **sensitive** data that you simply don't want to be permanent.

A really significant difference is that containers can't share tmpfs space unless they're running on Linux OS.

Two flags are used when creating tmpfs volume: tmpfs and mount.

Mount flag is newer and supports multiple options during containe startup.

Temporary filesystems are written to RAM (or to your swap file if RAM is filling up)

and not to the host or the container's own filesystem layer at Docker.com: Docker tmpfs.

```
[node1] (local) root@192.168.0.18 ~
$ docker run -d --name sql2 --mount source=v1,target=/app mysql:latest
e5dd11164c02682c247b247764fd63fecc683f6ec9de842b79649de5aea0c0f3
[node1] (local) root@192.168.0.18 ~
$ docker inspect sql2
[
```

```
        },
        "Mounts": [
            {
                "Type": "volume",
                "Name": "88dd3c0244f23baa7b776160fad8b40bf2af44a4a4e1a1558de745e275606beb",
                "Source": "/var/lib/docker/volumes/88dd3c0244f23baa7b776160fad8b40bf2af44a4a4e1a1558de745e
275606beb/_data",
                "Destination": "/var/lib/mysql",
                "Driver": "local",
                "Mode": "",
                "RW": true,
                "Propagation": ""
            },
```

Ex#2:

```
$ docker run -v /path/on/your/host:/data/db -d mongo
```

We can manage volumes through the docker cli directly (unlike bind mounts)

```
$ docker volume ls
DRIVER    VOLUME NAME
local     1981e0c33fca161d701d3fa5896c36970e45567f30bdbfbdf8cfb1be49b67a62
[node2] (local) root@192.168.0.17 ~
$ docker exec -it f4ccdc sh
# ls
bin   data  docker-entrypoint-initdb.d  home       lib    lib64   media  opt   root  sbin  sys  usr
boot  dev   etc                         js-yaml.js lib32  libx32  mnt    proc  run   srv   tmp  var
# cd data
# ls
configdb  db
# cd db
# ls
WiredTiger         _mdb_catalog.wt                    index-1--2331313126824442887.wt  mongod.lock
WiredTiger.lock    collection-0--2331313126824442887.wt  index-3--2331313126824442887.wt  sizeStorer.wt
WiredTiger.turtle  collection-2--2331313126824442887.wt  index-5--2331313126824442887.wt  storage.bson
WiredTiger.wt      collection-4--2331313126824442887.wt  index-6--2331313126824442887.wt
WiredTigerHS.wt    diagnostic.data                    journal
```

Practice question

1. Create the data in Ubuntu Image
2. Create the container with volume with name ubv1 and give path also where data will be stored.
3. Inspect the container running on ubuntu
4. Inspect the volume and path where data is stored.
5. Create one file inside the container in path specified with volume.
6. Delete the container.

7. Run one more container of ubuntu and check the data of previous container's volume.
8. If available ok , otherwise restore the data of first container volume.

Practice questions

1. How the files can be shared among 2 different containers?
2. How the connectivity of 2 containers can be checked?
3. How do you Filter the images by reference, where the tag is "latest."
4. How would you set the environment variables for the database connection details and application mode with help of run command?
5.

EX#3

- echo "<h1>message from Host</h1>" > ./target/index.html

- docker run -it --rm --name nginx -p 8080:80
  -v "$(pwd)"/target:/usr/share/nginx/html nginx